SmartDialer

Approach and implementation

A progressive and predictive outbound dialer for collections, with a Safety Controller that the pacing logic cannot bypass

Predictive vs progressive utilisation	Abandoned calls	Tests
44% → 67%	**0**	**79**
scenario A: 20% answer rate, 120s talk	predictive, scenarios A–D, provider A	unit, concurrency, recovery, safety, end-to-end

What this document covers. The problem and its constraints, the architecture, how correctness is guaranteed when many workers share one campaign, the agent and call state machines, the progressive dialer, the predictive pacing model, the Safety Controller and its mathematics, the telecom provider layer, every failure scenario from the brief with real output, simulation and load-test results, the scaling analysis, the architecture decisions, and the answer to the final question.

All numbers in this document come from runs of the submitted code: `python -m sim.run`, `python -m sim.failures`, `python -m loadtest.run` and `python -m pytest`.

Contents

1. Summary . . . 4

2. The problem and its constraints . . . 5

2.1 What must never happen . . . 5

2.2 What external systems do . . . 5

2.3 Scope . . . 5

3. Architecture . . . 6

3.1 Components . . . 6

3.2 Data model . . . 7

3.3 Why one database, and no queue or cache . . . 7

4. Correctness under concurrency . . . 8

4.1 Two workers see the same AVAILABLE agent . . . 8

4.2 One pacing leader per campaign, with fencing . . . 8

4.3 Ceilings are absolute levels, not deltas . . . 8

4.4 The distributed concerns from the brief . . . 8

5. State machines . . . 10

5.1 Agent . . . 10

5.2 Call . . . 10

5.3 Applying provider events . . . 11

6. Progressive dialer . . . 12

7. Predictive pacing . . . 13

7.1 The model . . . 13

7.2 Measuring the answer rate without bias . . . 13

7.3 Why 17 calls and not 10? . . . 13

8. The Safety Controller . . . 14

8.1 Assumptions . . . 14

8.2 Decisions and circuit breakers . . . 14

8.3 The boundary cannot be switched off by the pacer . . . 15

8.4 The 17-call proposal, decided . . . 15

8.5 When the answer rate changes suddenly . . . 15

9. Telecom providers and failure handling . . . 16

9.1 Provider interface . . . 16

9.2 Worker crash: reserved → initiated → crash . . . 16

9.3 Provider outage . . . 16

9.4 Agent availability suddenly drops (100 agents, 40 disappear) . . . 17

9.5 Duplicate and out-of-order events . . . 17

10. Simulation results . . . 18

10.1 Reading the results . . . 18

10.2 How much is the safety margin costing? . . . 18

11. Testing and performance . . . 19

11.1 Test suite (79 tests, about 20 seconds) . . . 19

11.2 Load test . . . 19

12. Scaling: what breaks first . . . 20

12.1 1,000 → 10,000 agents . . . 20

12.2 10,000 → 100,000 agents . . . 20

13. Architecture decisions . . . 21

13.1 Built, broken, fixed . . . 21

13.2 What I am least confident about . . . 22

13.3 With another week . . . 22

14. The final question . . . 23

Appendix A. Running the project . . . 24

Project layout . . . 24

1. Summary

Collections agents lose much of their day to dial attempts that never connect. A progressive dialer (one free agent, one call) is safe but wasteful: at a 20% answer rate an agent spends about half of every hour listening to rings. A predictive dialer starts calls before agents are free, but if more borrowers answer than there are agents, the surplus calls are **abandoned**, which for collections is a compliance issue, not just a bad experience.

This prototype implements both modes as one system. The central idea is that **predictive dialing is a bounded, revocable extension of progressive dialing**, not a separate mode:

- Every call is either **agent-bound** (agent reserved before dialing and handed to the provider, which connects the borrower on answer: as safe as progressive by construction) or **speculative** (agent claimed only when the borrower answers). Only speculative calls can be abandoned, and they are the only thing the predictive pacer can ask for.
- A separate **Safety Controller** sits between pacing and the provider. It ignores the pacer's model, uses an *upper* confidence bound on the answer rate, counts only agents the system controls right now, and caps speculative calls in flight so that P(more answers than agents) ≤ 2%. With no data the bound collapses to exactly progressive.
- The cap is enforced where it cannot be bypassed: an absolute ceiling checked in the same database transaction that creates each call, valid only for the current leadership epoch, carried by a permit only the Safety Controller can mint.
- The controller falls back to progressive on *evidence* that the model is wrong: an answer surge, agents disappearing, late webhooks, provider health, or the realised abandon rate. When in doubt it cancels unanswered calls, which costs a dial; it never risks an abandon, which is a compliance event.

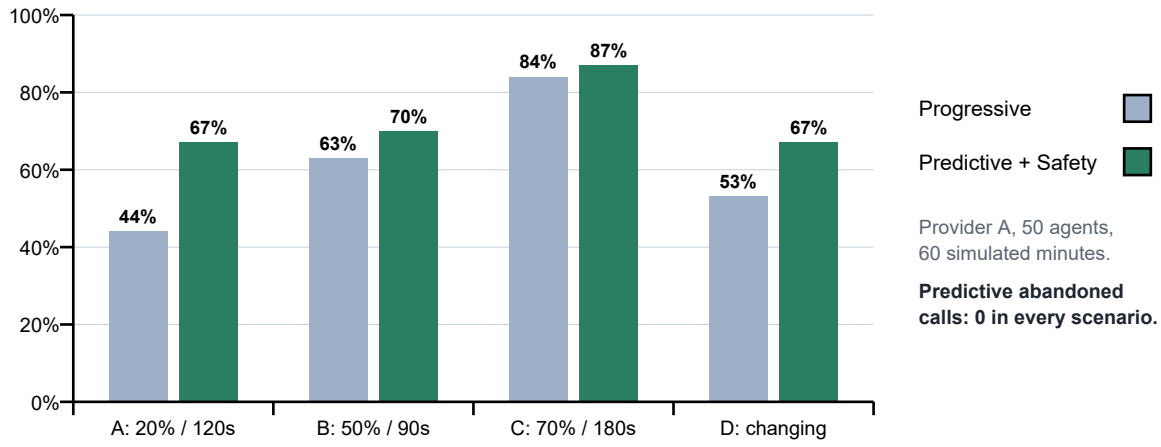


Figure 1. Share of logged-in agent time spent talking to borrowers.

Implementation in one paragraph
Single SQLite database as the only source of truth; every state change is a compare-and-set; partial unique indexes make double booking impossible at the data level; leases and a reconciler recover from any worker crash; a webhook inbox and a command outbox make provider events and side effects idempotent. No Redis, no Kafka, no cache: each would be a second source of truth with no problem it uniquely solves.

2. The problem and its constraints

The brief asks for a working prototype in either mode. This submission builds both, because the interesting engineering is in how they relate: the predictive mode is only acceptable if it inherits the safety of the progressive one.

2.1 What must never happen

Invariant	Why it matters	How it is enforced
One agent, at most one live call	Two workers reserving the same agent double-books a person	Compare-and-set reservation + partial unique index on live calls per agent
One borrower, at most one live call	Calling a borrower twice at once is a complaint and a compliance risk	Same pattern on borrowers + unique index
Answered borrower always gets a decision	Silence on an answered call is an abandon	ANSWERED is resolved to CONNECTED or ABANDONED in the same transaction
The pacer cannot switch safety off	A wrong model must not translate into abandons	Pacer returns a number; only the Safety Controller mints permits; ceiling re-checked in the database
A crash never strands an agent or borrower	Stuck agents are lost capacity; stuck borrowers are lost revenue	Leases on every in-flight responsibility; reconciler takes over

2.2 What external systems do

The brief is explicit that providers misbehave. The design assumes webhooks can arrive duplicated, out of order, late, or not at all; API calls can time out with an *unknown* outcome; a provider can go dark entirely; agents can vanish without logging out; and workers can crash at any line. Each of these is simulated, and each has a test (section 9).

2.3 Scope

- **Built:** both dialing modes, the Safety Controller, agent and call state machines, a provider interface with two behaviourally different mock providers (plus a scripted one for tests), crash recovery, a discrete-event simulator, a multi-process load test, 79 tests and the documentation set.
- **Not built:** a real telecom integration, calling-window and per-day regulatory rules, agent skills routing, and a user interface. These are listed under next steps.

3. Architecture

The brief suggests *Campaign → Pacing Engine → Safety Controller → Call Allocator → Telecom Provider*. The implementation follows that pipeline exactly and adds the three components a real system needs to survive the failures above: an event processor (webhook inbox), a command dispatcher (outbox) and a reconciler.

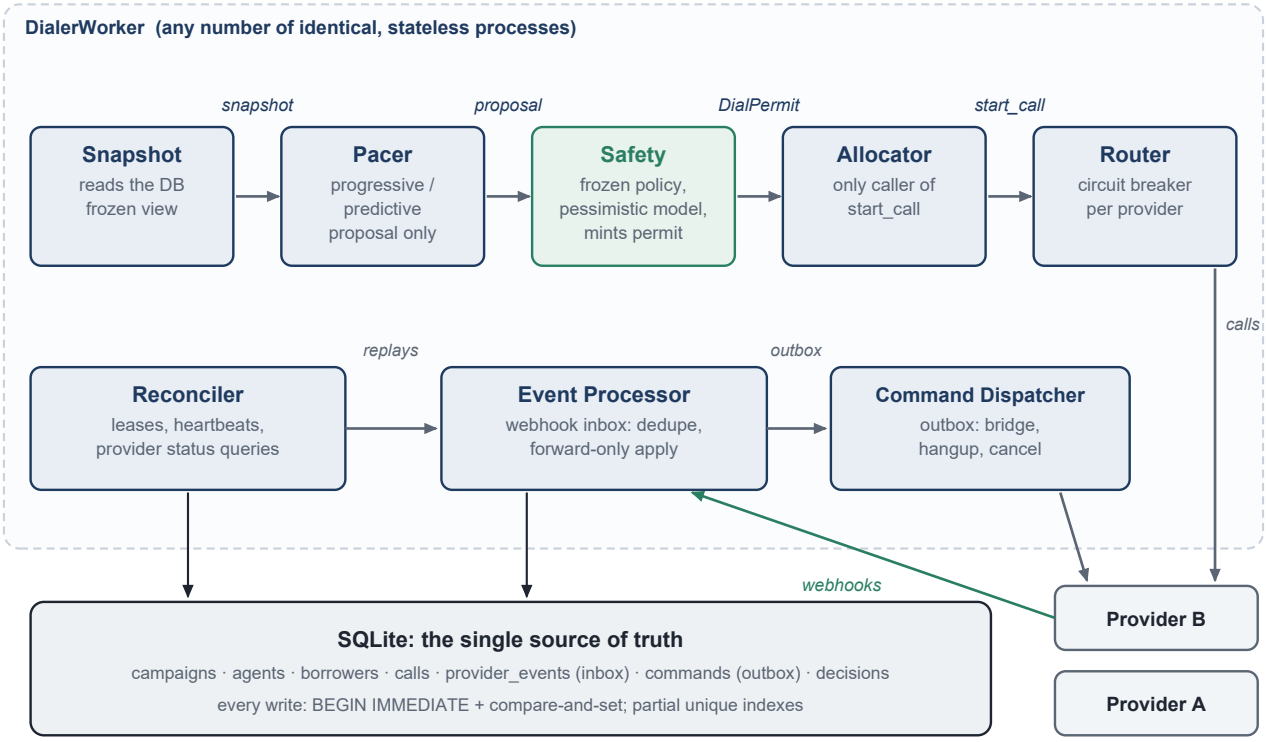


Figure 2. Components of one worker. Every worker is identical; all shared state is in the database.

3.1 Components

Module	Responsibility	Can it place a call?
pacing/progressive.py, pacing/predictive.py	Pure function: snapshot → proposal, with a written explanation of the number	No. Imports nothing that can reach the database, allocator or a provider (enforced by a test)
safety.py	Recomputes risk pessimistically; approves, reduces, rejects, falls back or sheds; mints a sealed DialPermit	No, it only authorises
allocator.py	Reserves agent + borrower + inserts the call atomically, then calls start_call outside the transaction	Yes, and only with a valid permit of the current epoch
events.py	Webhook inbox: dedupe by (provider, event_id); forward-only application	No
commands.py	Outbox: BRIDGE / HANGUP / CANCEL decided in a transaction, sent after commit, retried with backoff	Only acts on existing calls
recovery.py	Expired leases, lost heartbeats, stuck calls (asks the provider), pending events and commands	No new calls
agents.py	Agent desktop operations; what happens when an agent disappears	No
providers/	Interface, circuit breaker + router, mock providers A and B, scripted provider	n/a
db.py, repo.py, invariants.py	Schema; compare-and-set helpers; cross-table invariant checks	n/a

3.2 Data model

Table	Key columns	Role
campaigns	mode, leader, leader_until, epoch	Pacing leadership lease; epoch is the fencing token
agents	state, call_id, state_since, heartbeat_at, version	state_since orders 'longest idle first'
borrowers	status, priority, attempts, provider_failures, next_attempt_at, call_id	Borrower-caused and provider-caused failures counted separately
calls	id (also the idempotency key), kind, state, agent_id, owner, lease_until, next_check_at, timestamps, cancel_requested_at	Partial unique indexes: one live call per agent and per borrower
provider_events	PK (provider, event_id), status, outcome, lease_until	Inbox and dedupe in one table
commands	UNIQUE(call_id, kind), status, attempts, next_attempt_at	Outbox; idempotent enqueue
decisions	proposed, approved, action, explanation	Answers 'why did it dial 17 calls at 10:42:13?' after the fact

3.3 Why one database, and no queue or cache

Every hard question in the brief (can two workers take this agent? did this event already happen? who owns this half-finished call?) is a question about **shared state transitions**. A relational database with transactions answers all of them in one place: compare-and-set updates, unique indexes, an inbox table and an outbox table. Redis or Kafka would add a second source of truth and the consistency problems between the two without removing any of those questions. The interview question *"the database says AVAILABLE, the cache says RESERVED: which wins?"* cannot arise: there is no cache, and every state-changing decision is a compare-and-set against the database. If a cache were added for dashboards it would be advisory only.

4. Correctness under concurrency

4.1 Two workers see the same AVAILABLE agent

Reservation is a single statement inside a `BEGIN IMMEDIATE` transaction:

```
UPDATE agents SET state='RESERVED', call_id=:call, version=version+1
WHERE id = (SELECT id FROM agents
            WHERE campaign_id=:c AND state='AVAILABLE' AND heartbeat_at >= :fresh
            ORDER BY state_since LIMIT 1)          -- longest idle first
  AND state='AVAILABLE'                              -- the compare-and-set
RETURNING id
```

- **SQLite:** BEGIN IMMEDIATE takes the write lock before reading, so worker 2 waits until worker 1 commits, then sees RESERVED and either takes the next agent or gets no row.
- **Portable:** the `AND state='AVAILABLE'` guard keeps the code correct where transactions can overlap. In PostgreSQL at READ COMMITTED the second UPDATE blocks on the row lock, re-evaluates its WHERE clause after the first commits, and matches nothing. At scale it becomes `SELECT … FOR UPDATE SKIP LOCKED`.
- **Backstop in the data:** a partial unique index allows one live call per agent. Even a buggy path cannot create a second one; the insert fails and is counted as an integrity conflict.
- **Demonstrated:** 20 threads released by one barrier produce exactly one winner; 4 separate OS processes racing for 10 agents each win distinct agents.

4.2 One pacing leader per campaign, with fencing

Pacing reads a snapshot and decides a count, so two pacers on the same snapshot would both add N calls. Each campaign therefore has a leader lease (5s). The lease carries an **epoch** that increments whenever leadership changes hands, and the allocator rejects any permit whose epoch is not current, inside the reservation transaction. A leader that pauses (GC, network) past its lease and wakes up with an old permit is refused. Workers prefer their own partition of campaigns and take over another's only after its lease has been dead for a grace period, so leadership is sticky but still fails over in about 10 seconds.

4.3 Ceilings are absolute levels, not deltas

The Safety Controller does not say "add 15 calls"; it says "at most N agentless calls may be in flight". The allocator counts in-flight calls and inserts the new one in the same transaction, so even two valid permits computed from the same snapshot cannot add up to more than N.

4.4 The distributed concerns from the brief

Concern	How it is handled
Agent allocation	CAS on state + unique index; only agents with a fresh heartbeat are eligible
Borrower allocation	Same CAS on borrower status + unique index; highest priority, then earliest due
Duplicate jobs	One leader per campaign; absolute ceilings checked in-transaction
Retries	start_call idempotent on our call id; a retried borrower attempt is a new call row created only after the old one is terminal; outbox retries with backoff
Idempotency	Webhook PK dedupe + forward-only rule; UNIQUE(call_id, kind) for commands; reconciler replays provider status with deterministic event ids
Concurrent updates	Every write is CAS on the expected state, and for agents on the expected call_id
Worker crashes	Nothing important in memory: leases on call starts, inbox before apply, outbox for side effects; any worker's reconciler takes over
Stale state	Stale-heartbeat agents are ignored immediately; a new leader starts with an empty history and a warm-up; the snapshot is read in one transaction
Provider events	Inbox, dedupe, forward-only application, terminal states absorb everything, facts kept from stale events

5. State machines

Legal transitions are defined once, in `smartdialer/states.py`. Every state write goes through two helpers that (1) check the transition against those tables, raising on a programming error, and (2) use the expected current state as a compare-and-set guard, returning False if another worker got there first.

5.1 Agent

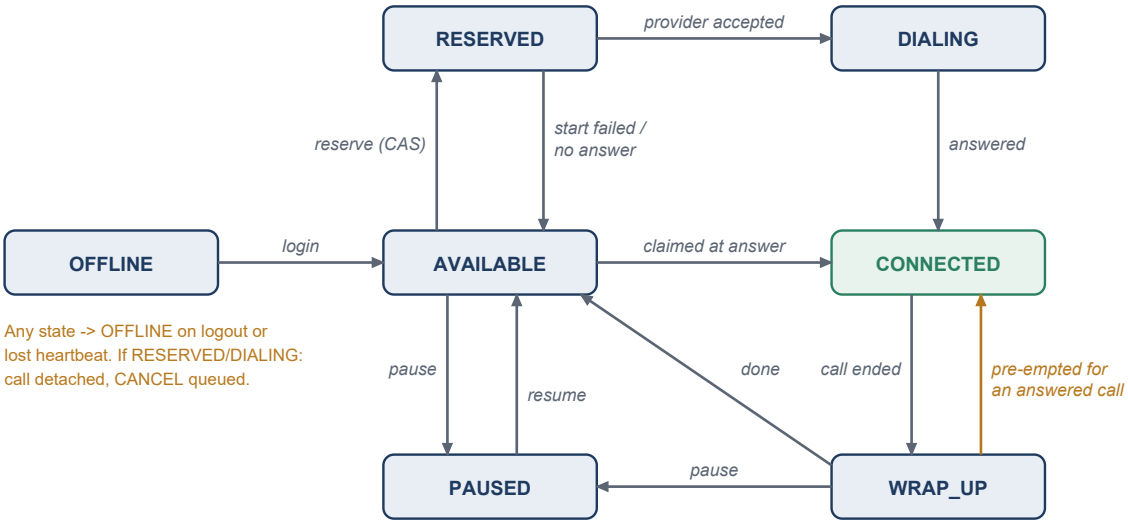
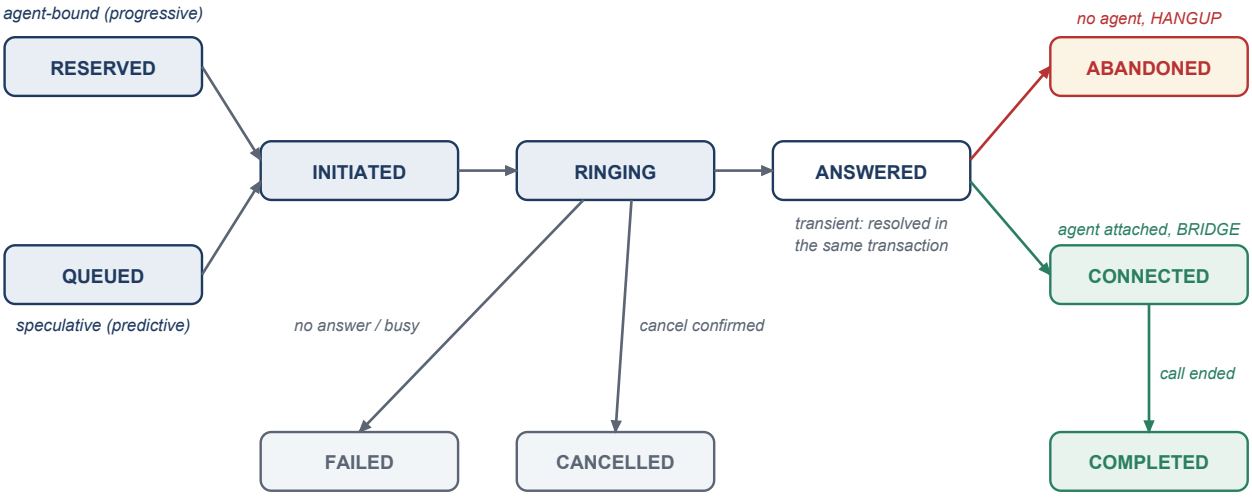


Figure 3. Agent lifecycle (main transitions).

Two transitions deserve explanation. **AVAILABLE → CONNECTED** is how a speculative call gets its agent: at the moment the borrower answers, the longest-idle available agent is claimed by CAS. **WRAP_UP → CONNECTED** pre-empts wrap-up when an answered borrower would otherwise be abandoned; this is also why the Safety Controller may count agents in wrap-up as capacity.

5.2 Call



Terminal states (COMPLETED, FAILED, CANCELLED, ABANDONED) absorb every later event.

Figure 4. Call lifecycle (main transitions).

- **RESERVED vs QUEUED.** RESERVED means an agent was claimed before dialing; its id is handed to the provider in start_call, so the provider bridges on answer without a round-trip through us. QUEUED means no agent yet.

- **ANSWERED is transient.** The transaction that applies ANSWERED also decides CONNECTED (agent attached, BRIDGE in the outbox) or ABANDONED (HANGUP in the outbox). There is no moment where an answered borrower exists without a decision.
- **Cancel is a request.** cancel_requested_at is set and a CANCEL queued; the call becomes CANCELLED only when the provider confirms. Until then it can still be answered, and it still counts as risk.

5.3 Applying provider events

Rule	Why
Insert into provider_events keyed by (provider, event_id); repeat = DUPLICATE	Redelivered webhooks
Only move a call forward by rank; otherwise record as STALE	Out-of-order delivery and re-sends under new ids
Terminal states absorb everything	COMPLETED, then ANSWERED, then RINGING
Agent moves are CAS on (state, call_id)	A late event for an old call cannot move an agent now on a new call
Facts survive staleness (a late ANSWERED still fills answered_at)	Accurate answer-rate statistics
COMPLETED (reason NO_AGENT) before ANSWERED seen → ABANDONED	The borrower answered and got no agent: count it honestly
RINGING / ANSWERED on a still-RESERVED call clears the start lease	The provider has the call even if the worker that sent it died
Ingest and apply are separate transactions; apply retried from the inbox	A crash between received and applied loses nothing
A deterministic failure on retry is parked as ERROR	One poison webhook must not stop every worker

The sequences named in the brief, as produced by `python -m sim.failures`:

```
ANSWERED evt-42                     -> CONNECTED  call=CONNECTED version=2 agent=CONNECTED
ANSWERED evt-42 (redelivery)        -> DUPLICATE  call=CONNECTED version=2 agent=CONNECTED
ANSWERED evt-43 (re-sent, new id)   -> STALE      call=CONNECTED version=2 agent=CONNECTED
COMPLETED evt-44                    -> APPLIED    call=COMPLETED version=3 agent=WRAP_UP

COMPLETED             -> APPLIED  call=COMPLETED/NORMAL  agents[a0=AVAILABLE]  borrower=DONE
ANSWERED (90s late)   -> STALE
RINGING (90s late)    -> STALE
```

6. Progressive dialer

The progressive pacer proposes one agent-bound call per AVAILABLE agent with a fresh heartbeat, capped by the borrowers that are due. The questions the brief asks it to decide:

Decision	Implementation
Which borrower	Highest priority, then earliest next_attempt_at, among PENDING borrowers that are due (CAS claim)
Which agent	Longest idle: ORDER BY state_since, among AVAILABLE agents with a fresh heartbeat (CAS reservation)
When the agent becomes reserved	In the same transaction that claims the borrower and inserts the call, before any network I/O
What happens when the call fails	Agent back to AVAILABLE in the same transaction. No answer / busy: borrower attempt +1, retry later, EXHAUSTED after max attempts. Provider error: provider_failures +1, quick retry, not charged as an attempt
The agent disappears during setup	Agent OFFLINE; call detached; CANCEL requested. If the borrower answers first, another agent is claimed; only if none is free is the call hung up and counted as abandoned

The allocator never holds a transaction across network I/O. Its sequence for one call:

```
tx 1   check fencing epoch                (stale leaders refused)
       reserve agent       AVAILABLE -> RESERVED          (CAS)
       claim borrower      PENDING   -> IN_PROGRESS       (CAS)
       insert call RESERVED with lease: owner = this worker, lease_until = now + 10s
----   re-read the call: if a cancel was requested meanwhile, close it and never dial
----   provider.start_call(call_id, phone, agent_id)    -- call_id is the idempotency key
tx 2   RESERVED -> INITIATED; agent RESERVED -> DIALING; clear the lease
```

The 50-agent guarantee
If 50 agents are available, the system cannot create more than 50 agent-bound calls: each call needs a successful CAS on a distinct agent, and the database refuses a second live call for any agent. Repeated ticks do not add calls for busy agents (test_50_available_agents_never_produce_more_than_50_calls).

7. Predictive pacing

The pacer answers "how many calls *should* start now so that answers line up with agents becoming free?". It optimises utilisation and is allowed to be wrong; whether its proposal is *allowed* is the Safety Controller's question.

7.1 The model

```
p         answer rate: last 60s, using the last 5 min as its prior, which uses a campaign prior
T         mean dial -> answer time
capacity  agents free now
          + agents in wrap-up expected to finish within T
          + talking agents expected to finish talk + wrap-up within T  (exponential talk time)
expected  un-answered agentless calls already in flight x p
proposal  floor( (capacity - expected) / p )      capped by a max ratio and by borrowers due
```

7.2 Measuring the answer rate without bias

Answer rates are measured on **mature dial cohorts**: calls dialed in a window that ended at least 40 seconds ago, so every outcome is known. The obvious alternative, counting attempts that *resolved* recently, is biased: answers resolve 3–15s after dialing, no-answers only at the 30s ring timeout, so whenever the dial rate rises the answer rate looks inflated. The first version of this code had exactly that bug; at a constant 20% answer rate it reported jumps to 35% and triggered false alarms.

7.3 Why 17 calls and not 10?

Every decision stores its explanation. A worked example (also a unit test):

```
pacer[predictive] proposed 17 (bound=0, speculative=17):
    answer rate p=0.29 (60s: 14/50, 5m: 120/400, prior 0.30)
    dial->answer ~15s; talk ~150s; wrap ~10s
    agents free within 15s: 3 available + 2.0 of 2 in wrap-up + 1.5 of 45 talking = 6.5
    5 un-answered calls in flight x p = 1.4 answers expected
    missing 5.0 answers / p = 17.5 -> 17 calls
```

So 17 rather than 10: at a 29% answer rate about 3.5 dials produce one answer, and about 5 answers are missing. Ten calls would leave roughly two agents idle for another cycle. But the pacer does not decide; section 8.4 shows what the Safety Controller does with this proposal.

8. The Safety Controller

The controller recomputes everything from the same snapshot with its own, deliberately pessimistic assumptions. It does not trust the pacer's model; if that model is wrong, the pacer cannot be the component that notices.

8.1 Assumptions

- **Answer rate:** the Wilson *upper* bound (z = 1.96) over the long and short windows, whichever is higher, raised further to the share of recent dials that are *already answered*, which is a hard lower bound on today's rate because answers can only increase. With no data the bound is 1.0.
- **Capacity:** agents AVAILABLE now plus agents in WRAP_UP (which can be pre-empted). **Agents who are talking are never counted**: the system does not control when they finish. This is the main difference from the pacer and the main source of conservatism.
- **Ceiling:** the largest number n of un-answered agentless calls such that P(Binomial(n, p_{ucb}) > capacity) ≤ 2%.

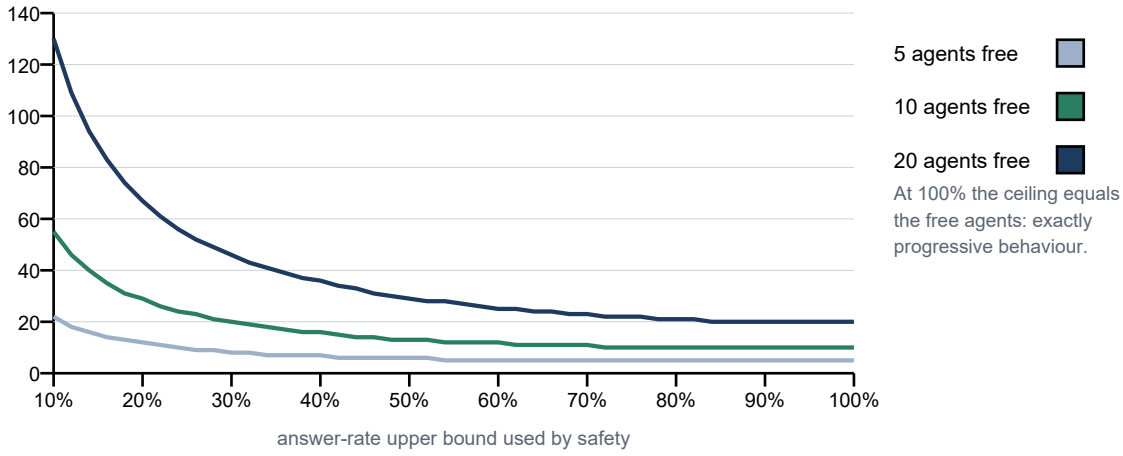


Figure 5. The ceiling the controller allows, computed with the project's own code. Low answer rates allow several calls per free agent, which is where the utilisation gain comes from; as the bound approaches 100% the ceiling converges to the number of free agents.

8.2 Decisions and circuit breakers

Condition	Action
Provider circuit OPEN	REJECT everything; calls in flight are left alone and reconciled later
Provider HALF_OPEN	One probe call per tick
Fewer than 50 resolved attempts; leader younger than 10s	FALLBACK_PROGRESSIVE
Realised abandon rate (30 min) ≥ 3%	FALLBACK for a 60s cool-down; ≥ 1.5% halves the tolerance
> 0.5% of webhooks arrive more than 3s late	FALLBACK: an answered agentless call could not be bridged before the borrower gives up
Logged-in agents fall ≥ 15% within 10s	FALLBACK + SHED (cancel not-yet-answered agentless calls)
Already-answered share of recent dials is significantly above the bound	FALLBACK + SHED down to progressive's invariant (agentless in flight ≤ free agents)
Answer rate 60s vs 5 min differs ≥ 10 points with z ≥ 3.5	FALLBACK for a 60s cool-down
Otherwise	APPROVE the proposal, or REDUCE it to the room under the ceiling (REJECT if there is none)

In FALLBACK_PROGRESSIVE, agents needed to absorb agentless calls still in flight are held back, and only the rest are reserved for new agent-bound calls. Shedding happens only after a shock: capacity consumed by our own answers was priced in when the calls were dialed, and cancelling on it just churns calls (an early version cancelled 230 calls in 15 minutes that way).

8.3 The boundary cannot be switched off by the pacer

- The pacer receives a frozen snapshot and returns a frozen proposal. The pacing package imports nothing that can reach the database, the allocator or a provider; a test parses its imports and fails if that ever changes.
- The policy is a frozen dataclass handed to the controller by the composition root; nothing passes it to the pacer, and it cannot be mutated.
- The allocator accepts only a DialPermit, which only the controller can construct. It carries the leadership epoch and a 2-second expiry, and its ceiling is re-checked atomically in the database.
- Honest limit: in-process Python cannot stop determined code from importing a private name. In production the Safety Controller and allocator would be their own service and the permit a signed token.

8.4 The 17-call proposal, decided

```
safety: REDUCE -> bound=0, speculative=1, shed=0 (predictive)
    capacity 5 = 3 available + 2 in wrap-up (talking agents never counted); answer-rate upper bound 0.42
    ceiling 6 agentless calls keeps P(answers > 5) <= 2.0%; 5 in flight -> room for 1
    proposal 17 cut to 1
```

The controller counts only the 5 agents it controls, uses p = 0.42 rather than 0.29, and already has 5 calls in flight. With 8 agents available instead of 3, the pacer asks for 34 and safety allows 10.

8.5 When the answer rate changes suddenly

70% → 10% is the safe direction. Fewer borrowers answer than planned, so agents wait; nothing is abandoned. Mature cohorts drift towards 10% and the shift detector forces a progressive cool-down while the model is untrustworthy. Utilisation dips; the abandon rate does not move.

10% → 45% is the dangerous direction (scenario D), because calls dialed for a 10% world are answered at 45%. Three layers handle it: the ceiling uses the upper bound, never the estimate; within seconds the already-answered share of recent dials exceeds that bound, which is assumption-free proof that the model is wrong, so the controller stops dialing ahead and cancels unanswered calls; and the realised abandon rate trips the 3% breaker as a last resort. Before the second layer existed, scenario D abandoned 44 calls (3.45%) in the minute after the surge. With it, 0 on four of five seeds and 2 (0.16%) on the fifth.

9. Telecom providers and failure handling

9.1 Provider interface

```
class TelecomProvider(Protocol):
    def start_call(self, call_id, phone, agent_id=None) -> str   # idempotent on call_id
    def cancel_call(self, call_id) -> None      # unknown id -> remembered (tombstone)
    def hangup(self, call_id) -> None
    def bridge(self, call_id, agent_id) -> None # idempotent
    def get_call_status(self, call_id) -> ProviderCallStatus | None

ProviderTimeout   outcome unknown: the request may or may not have taken effect
ProviderRejected  definitive: the request did not take effect
```

The two exception types matter: a timeout on start_call does not mean the call was not placed. The dialer treats it as ambiguous, keeps the reservation and asks the provider later. A circuit breaker per provider (HEALTHY, DEGRADED, HALF_OPEN, OPEN) feeds the Safety Controller, and a router sends new calls to the healthiest provider.

	Provider A	Provider B
Setup time	0.5–1.5s	2–6s
Webhook delivery delay	0.05–0.2s	0.2–3s, and 3% of webhooks 8–30s late
start_call failures	0.5% rejected	2% rejected, 3% time out (half of those were placed anyway)
Duplicate webhooks	none	15% (half same id, half re-sent under a new id)
Behaviour of the dialer	Predictive used	Safety detects late webhooks and stays progressive

Both mocks model what makes abandonment real: an answered borrower who is not bridged to an agent within 5 seconds hangs up. A lost bridge request, a late ANSWERED webhook or an outage at the moment of answer all produce genuine abandoned calls in the simulation.

9.2 Worker crash: reserved → initiated → crash

The call row carries a lease until the provider's acceptance is recorded. When the system comes back, any worker's reconciler finds the expired lease and asks the provider about that call id. If the provider has it, the call is **adopted** (INITIATED, agent DIALING) and its status replayed as events. If not, the id is **cancelled first** (a tombstone, so a delayed start can never create it), then the agent and borrower are released. If other workers are alive, the provider's RINGING webhook usually arrives first and itself confirms the start.

```
t=+0s   worker crashed at 'after_start_call' (provider accepted the call, our DB was not updated)
        call=RESERVED  agents[a0=RESERVED, a1=AVAILABLE]  borrower=IN_PROGRESS
t=+11s  restarted worker: lease expired -> provider has the call -> adopted
        call=INITIATED agents[a0=DIALING, a1=DIALING]; start_call sent exactly 1 time
t=+15s  ANSWERED  -> CONNECTED
t=+75s  COMPLETED -> COMPLETED/NORMAL  agents[a0=WRAP_UP]  borrower=DONE
```

ANSWERED, crash, then COMPLETED: the webhook is committed to the inbox before it is applied. A crash before or during application leaves it PENDING; when its lease expires any worker applies it, and a COMPLETED arriving meanwhile is applied on its own merits. A crash after commit but before the bridge request leaves the request in the outbox, and the next reconciler pass sends it.

9.3 Provider outage

Question from the brief	What happens
Existing calls	Never released early. If we cannot learn whether a borrower answered, we must assume they may be on the line (for an agent-bound call, possibly already talking to the agent). Held webhooks and status queries settle everything on recovery; after 15 minutes a call is closed as UNRESOLVED and flagged for review
New calls	Timeouts open the circuit breaker within about 10s; afterwards safety REJECTs every tick (173 of 180 decisions in the demo). With a second provider, traffic fails over: 38 calls went to the backup during the outage
Retries	Provider-caused failures reschedule the borrower soon, counted as provider_failures, not as borrower attempts. Outbox commands back off exponentially
Pacing	REJECT while OPEN, one probe per tick when HALF_OPEN, then progressive until the late-webhook signal clears (the held webhooks arrive minutes late)
Cost	4 speculative calls answered during the outage could not be bridged (0.73% for the run). Agent-bound calls were unaffected, because the provider bridges them itself

9.4 Agent availability suddenly drops (100 agents, 40 disappear)

```
-- agents log out explicitly
   t= 899.5   logged-in 100  available 24  agentless-unanswered 76  REDUCE                shed 0
   t= 900.5   logged-in  60  available 12  agentless-unanswered 81  FALLBACK_PROGRESSIVE   shed 31
   reaction 0.5s; 31 not-yet-answered calls cancelled; abandoned in the next 3 minutes: 0
-- desktops die silently (heartbeat timeout 10s)
   t= 909.5   logged-in  60  available 16  agentless-unanswered 83  FALLBACK_PROGRESSIVE   shed 23
   reaction 9.5s; 23 calls cancelled; abandoned in the next 3 minutes: 0
```

The dialer reacts on the next tick to anything it is told about, and within the heartbeat timeout to silent disappearance; the snapshot ignores a stale heartbeat even before the reconciler logs the agent out. The heartbeat timeout is therefore the most important knob for this failure.

9.5 Duplicate and out-of-order events

Duplicates do not create extra transitions: the same id is rejected by the inbox primary key, the same meaning under a new id by the forward-only rule. With 50% of webhooks delivered twice for 15 simulated minutes, no call exceeded its 4 legitimate transitions. Out-of-order events do not break anything: terminal states absorb them and late events still contribute facts. For speculative calls a late ANSWERED is a real problem (we cannot bridge what we have not seen), which is exactly why webhook lateness is a Safety Controller input.

10. Simulation results

The simulator drives the real worker code (SQLite schema, allocator, Safety Controller, event processor, reconciler) against the mock providers on a simulated clock: one hour of dialing for 50 agents takes about 4 seconds and is reproducible from a seed. Scenarios A–C are the brief's; D changes answer rate (30% → 70% → 10% → 45%, every 15 minutes) and talk time (90 → 180 → 60s, every 20 minutes).

Sc	Pv	Mode	Util	Wait	Started	Conn	Ab	Ab %	Safety decisions (one per second)
A	A	progressive	44%	52%	3537	718	0	0.00%	APPROVE 3600
A	A	predictive	67%	27%	5190	1037	0	0.00%	REJECT 1926, REDUCE 1617, FALLBACK 41, APPROVE 16
A	B	progressive	41%	55%	3171	665	0	0.00%	APPROVE 3600
A	B	predictive	41%	55%	3171	665	0	0.00%	FALLBACK 3600
B	A	progressive	63%	30%	2640	1276	0	0.00%	APPROVE 3600
B	A	predictive	70%	21%	3097	1508	0	0.00%	REJECT 1783, REDUCE 1776, FALLBACK 41
B	B	progressive	59%	34%	2454	1214	0	0.00%	APPROVE 3600
B	B	predictive	59%	34%	2454	1214	0	0.00%	FALLBACK 3600
C	A	progressive	84%	11%	1258	876	0	0.00%	APPROVE 3600
C	A	predictive	87%	8%	1418	965	0	0.00%	REJECT 2306, REDUCE 1119, APPROVE 134, FALLBACK 41
C	B	progressive	82%	14%	1203	826	0	0.00%	APPROVE 3600
C	B	predictive	79%	16%	1323	875	1	0.11%	FALLBACK 3584, REJECT 10, REDUCE 6
D	A	progressive	53%	41%	3113	980	0	0.00%	APPROVE 3600
D	A	predictive	67%	26%	4476	1249	0	0.00%	REJECT 1500, REDUCE 1394, FALLBACK 694, APPROVE 12
D	B	progressive	50%	45%	2831	932	0	0.00%	APPROVE 3600
D	B	predictive	50%	45%	2831	932	0	0.00%	FALLBACK 3600

Sc = scenario, Pv = provider, Conn = connected, Ab = abandoned. Utilisation = share of logged-in time talking; waiting = share AVAILABLE, RESERVED or DIALING. REJECT almost always means the pacer wanted more but the in-flight ceiling was already reached.

10.1 Reading the results

- **The gain is largest where progressive hurts most.** At a 20% answer rate progressive agents spend over half their time waiting; predictive cuts that by about half and lifts talk time from 44% to 67%. At 70% there is little left to gain (84% → 87%).
- **Zero abandons with provider A** in every scenario, including D's surges. Across five seeds per scenario (20 runs) there were 2 abandoned calls in total, both in one run of scenario D (0.16%).
- **Provider B is handled by refusing to dial ahead.** Its webhooks are too late to bridge speculative calls reliably, so safety stays in FALLBACK_PROGRESSIVE for the whole run: the same utilisation as progressive, at most 1 abandon per hour (0.11%).

10.2 How much is the safety margin costing?

Overflow tolerance	0.5%	2% (default)	5%	10%	20%
Utilisation (scenario A)	65%	67%	68%	69%	73%
Abandoned calls	0	0	0	0	0

Abandon breaker disabled to show the raw trade-off. Even 20% produced no abandons here, because the bound is conservative in other ways (upper-bound answer rate, talking agents never counted). The 2% default is deliberately cautious.

11. Testing and performance

11.1 Test suite (79 tests, about 20 seconds)

File	Tests	What it proves
test_concurrency.py	8	20 threads / 4 processes, one agent: one winner; unique indexes; stale leader fenced off; two permits cannot exceed a ceiling; leadership partitioning; globally unique provider ids
test_events.py	14	Duplicates, ANSWERED×3 + COMPLETED, COMPLETED/ANSWERED/RINGING, crash before/during/after applying ANSWERED, poison events, abandon accounting
test_recovery.py	12	Crash after start_call (adoption), crash before it (tombstone), ambiguous timeouts, lost webhooks, provider outage end to end
test_progressive.py	11	50 agents → never more than 50 calls; failures release agents; agent disappears during setup; heartbeat loss
test_safety.py	14	Bound math; fallback triggers; shedding on agent drop; surge detection; permits cannot be forged; policy immutable; pacing import boundary
test_pacing.py	10	Binomial and Wilson math against brute force; the 17-call worked example
test_simulation.py	10	End-to-end runs and all seven failure demos

11.2 Load test

`python -m loadtest.run --sweep` runs real worker processes on one SQLite file (WAL mode) with sub-second mock calls, 2,000 agents, each worker leading its own campaign:

Workers	Calls/s	Webhooks/s	Write txns/s (per call)	Tick p50 / p95 / p99 ms	Lock errors	Invariants
1	590	1,367	4,730 (8.0)	77 / 105 / 134	0	OK
2	547	1,212	4,253 (7.8)	139 / 227 / 254	0	OK
4	603	1,276	4,533 (7.5)	124 / 553 / 1,480	0	OK
8	625	1,338	4,918 (7.9)	8 / 186 / 1,387	0	OK

- **Correctness held under contention:** no double bookings, no lock errors, every invariant intact.
- **Throughput is flat at about 600 calls/s** (about 4,700 small write transactions per second) from one process to eight. That is one SQLite file's single writer.
- **The cost of more processes is latency:** they queue on the write lock and p99 tick time climbs from about 130ms to 1.4s. A slow tick is a slow reaction to an agent drop or a surge, so beyond this point the database starts to hurt safety responsiveness.

Agents in one campaign	Borrowers	Snapshot per tick	Reservation txn
100	2,000	0.5 ms	0.06 ms
1,000	20,000	1.8 ms	0.08 ms
10,000	200,000	32 ms	0.09 ms
50,000	1,000,000	183 ms	0.15 ms

Reservation cost is flat (indexed compare-and-set). The pacing snapshot scans recent call history, so it grows linearly with campaign size.

12. Scaling: what breaks first

At a 20% answer rate and 2-minute talk time an agent generates about 0.03 dial attempts per second, each about 8 write transactions and 3–4 webhooks.

Agents	Dials/s	Write txns/s	Webhooks/s	First thing to break
100	3	25	10	Nothing; one worker uses about 0.5% of measured capacity
1,000	30	250	100	Nothing structural; a second worker is for availability
10,000	300	2,500	1,000	The allocator's sequential start_call, then the per-tick snapshot
100,000	3,000	25,000	10,000	The single database writer (about 5x over)

12.1 1,000 → 10,000 agents

- **Sequential start_call breaks first.** The mock answers instantly; a real provider takes 100–500ms per API call. At 300 dials/s that is 30–150 seconds of waiting per second, from one leader. Fix: keep the reservation transactions as they are, but issue start_call concurrently from a bounded pool. The permit and in-transaction ceiling already make that safe.
- **Snapshot cost** (32ms at 10k agents, linear in history). Fix: maintain rolling counters in a campaign_stats row updated by the transaction that applies each event, so the snapshot is O(1).
- **One leader per giant campaign.** Fix: split it into agent pools of a few hundred, each with its own leader, ceiling and Safety Controller.

12.2 10,000 → 100,000 agents

- **PostgreSQL** with row locks and SELECT … FOR UPDATE SKIP LOCKED. The SQL is already written compare-and-set style for READ COMMITTED, so this is a driver change rather than a redesign.
- **Shard by campaign or agent pool.** A pool's agents, borrowers and calls never interact with another pool's, so the data partitions cleanly; each shard is an independent instance of this design.
- **Webhook ingestion** at about 10,000/s: batch inbox inserts and route by call-id prefix. Only here would a log such as Kafka earn its place, as a burst buffer in front of the inbox, not as a replacement for its dedupe.
- **Carrier limits become the real ceiling:** calls-per-second caps and trunk concurrency. The Safety Controller should hold per-provider CPS budgets the same way it holds the abandonment budget.

> **"We just went from 1,000 to 100,000 agents. What breaks first?"**
> In this prototype: the sequential start_call loop of each campaign's pacing leader, because it waits on the network for every call. Then the single SQLite writer at about 5,000 transactions per second, around 15–20 thousand agents. The reservation logic itself is not a bottleneck at any of these sizes.

13. Architecture decisions

Decision	Why	What it makes harder
Python, standard library only	The hard parts are semantics, not speed; runs with pip install pytest	CPU-bound math at very large scale; no async start_call yet
One relational DB as the only source of truth	Transactions, CAS and unique indexes answer every shared-state question in one place	SQLite has one writer (~5,000 txns/s) and one host; next step is PostgreSQL, then sharding
No Redis, Kafka or cache	Each is a second source of truth with consistency bugs between the two	Read scaling (use replicas); webhook bursts at 100k+ agents
Leases + reconciler	A crash at any line leaves recoverable rows, with no coordinator to fail	Recovery time is bounded by lease length
Leader per campaign + fencing epoch; absolute ceilings	No duplicate pacing; stale leaders refused; permits cannot add up	One allocation loop per campaign (split into pools)
Inbox for webhooks, outbox for side effects	Closes the crash windows between received/applied and decided/sent	About 8 write transactions per call
Agent-bound vs speculative calls	Progressive stays safe even with late webhooks; risk isolated to a quantity we can bound	Two answer paths; provider must support connect-on-answer
Separate, pessimistic Safety Controller	A wrong model cannot be the one to notice it is wrong	Leaves utilisation on the table, deliberately
Hold, don't release, when the provider is silent	Never double-book a person who may be on a call	Agents held for the length of an outage
Simulate on the real code	Results are about this code, reproducible from a seed	The mock world is simpler than reality

13.1 Built, broken, fixed

The brief asks for *Think → Design → Build → Break → Fix → Explain*. The simulator, failure demos and two code-review passes found real defects; each was fixed and now has a test or demo check:

Found	Fix
Progressive abandoned calls when webhooks were late	Hand the reserved agent to the provider in start_call: it bridges on answer by itself
Shedding cancelled 230 calls in 15 minutes	Shed only after a shock; normal consumption of capacity was already priced in
Predictive on provider B abandoned ~2%	Late-webhook gate: refuse to dial ahead through a provider whose webhooks are late
Scenario D surge: 44 abandons (3.45%)	Already-answered cohort bound + surge shedding down to progressive's invariant
Answer rate biased upward by resolution time	Measure on mature dial cohorts
Outage: frozen calls filled the ceiling and blocked failover	Calls older than the longest possible ring stop counting as risk
A webhook overtaking a crashed allocator left its lease behind	RINGING/ANSWERED on a RESERVED call confirm the start and clear the lease
One impossible event blocked every worker forever	Correct mapping, plus parking of deterministic failures
Borrowers already reached were re-dialed	COMPLETED marks the borrower DONE even if ANSWERED was never seen
Mock provider ids collided across processes; webhooks dropped	Globally unique provider call ids (load-test throughput 66 → 263 calls/s)

13.2 What I am least confident about

- **Silent agent disappearance:** for up to the heartbeat timeout a dead desktop still counts as available. The real fix is a softphone or SIP registration signal.
- **Calls live when a provider dies:** answered speculative calls that cannot be bridged are abandoned, and nothing can prevent that once the provider is down; only dialing ahead less through unreliable providers, and failing over fast, reduce it.
- **The lateness gate is evidence-based:** a provider with late webhooks gets a few minutes of predictive dialing before enough lateness is observed. Persisting each provider's profile across restarts would close that window.
- **In-process permit sealing** is enforced by structure and tests, not by the language; production would use a separate service and signed permits.

13.3 With another week

- Concurrent start_call with a bounded pool, and the incremental campaign_stats row.
- PostgreSQL behind the same SQL, and a load test across real hosts.
- A real provider (Plivo): answer-URL bridging for agent-bound calls and webhook signature verification.
- Persistent per-provider lateness and health profiles.
- Calling windows, per-day attempt limits and callback-after-abandon rules inside the Safety Controller.

14. The final question

> How would you build a SmartDialer that gets as much of the utilisation benefit of predictive dialing as possible, while retaining the deterministic safety characteristics of progressive dialing?

Make predictive a **bounded, revocable extension of progressive**, not a separate mode.

1. Keep two kinds of call

An agent-bound call has its agent reserved before dialing and handed to the provider, which connects the borrower on answer without asking us: it is as safe as progressive by construction. A speculative call is the only thing that can be abandoned, so it is the only thing the predictive engine is allowed to ask for.

2. The safety bound belongs to safety, not to the model

Speculative calls in flight are capped so that P(more answers than agents) ≤ 2%, using the *upper* confidence bound of the answer rate and only agents the system controls now, never agents still talking. With no data the bound is 1.0 and the system is exactly progressive. The utilisation comes from dialing several calls per free agent when answer rates are low, which is precisely where progressive wastes the most agent time.

3. Enforce it where it cannot be bypassed

The pacer can only return a number. The cap is an absolute level checked in the same transaction that creates each call, valid only for the current leadership epoch, so stale leaders and parallel workers cannot add up to more.

4. Fall back the moment the model is wrong, and keep an exit open

Revert to progressive on evidence, not averages: an answer surge, agents disappearing, late webhooks, provider health, or the realised abandon rate. Cancelling a call that has not been answered costs a dial; abandoning one that has is a compliance event. So when safety is in doubt, it cancels.

> **Outcome**
> In the simulator: 67% utilisation against 44% at a 20% answer rate, with zero abandoned calls. With a provider whose webhooks are too slow to bridge safely, it declines to dial ahead and behaves exactly like progressive.

Appendix A. Running the project

Requires Python 3.11 or later; the only dependency is pytest.

```
python -m pip install -r requirements-dev.txt

python -m pytest                    # 79 tests, about 20s
python -m sim.run                   # scenarios A-D x provider A/B x both modes (about 90s)
python -m sim.run --scenario D --mode PREDICTIVE --provider A --explain 4
python -m sim.failures              # every failure case, step by step (about 15s)
python -m sim.failures crash        # one of: race crash outage drop duplicates order setup
python -m loadtest.run              # 4 worker processes on one database, real time
python -m loadtest.run --sweep      # 1 / 2 / 4 / 8 workers
python -m loadtest.run --scale      # per-tick cost vs campaign size
```

Project layout

```
smartdialer/
  states.py        agent and call state machines (the only definition of legal transitions)
  db.py, repo.py   schema; compare-and-set helpers every state change goes through
  pacing/          progressive and predictive pacers: pure functions, propose only
  safety.py        Safety Controller, SafetyPolicy (frozen), DialPermit (sealed)
  allocator.py     the only caller of provider.start_call
  events.py        webhook inbox: dedupe and forward-only application
  commands.py      outbox: bridge / hangup / cancel with retries
  recovery.py      reconciler: leases, heartbeats, provider status, pending work
  agents.py        agent desktop operations, agent disappearance
  observe.py       builds the pacing snapshot from the database
  worker.py        composition root: one tick of one worker
  providers/       interface, circuit breaker + router, mock providers A/B, scripted provider
  invariants.py    cross-table invariants checked everywhere
sim/               discrete-event simulator, scenarios A-D, failure demos
loadtest/          multi-process load test, process race
tests/             unit, concurrency, recovery, safety, end-to-end
docs/              architecture, state machines, pacing, failure scenarios, results, scaling,
                   decisions
```

Size: about 2,950 lines of dialer code, 1,200 lines of simulator and load test, and 1,000 lines of tests.